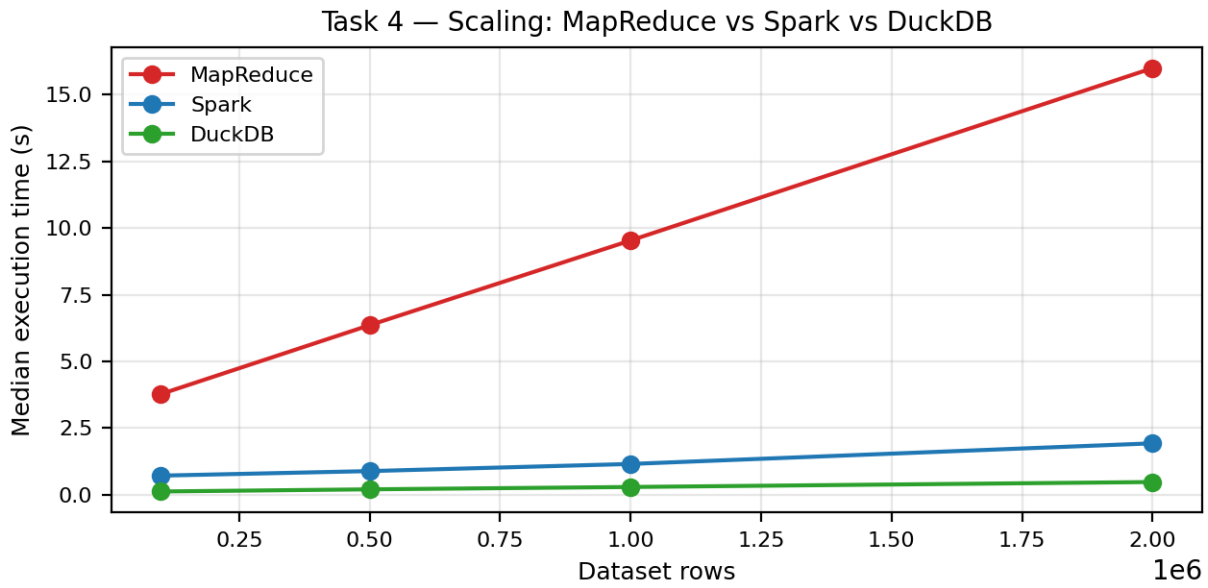
Observations — Big Data Analytics Assignment 1 (Student ID: 2023341)

MapReduce vs Apache Spark vs DuckDB — NYC Yellow Taxi, January 2026. Machine: Intel Core i3-1115G4 (2 cores/4 threads), 8GB RAM, Linux Mint 22.3 (Live USB, storage type recorded as Other/Live USB — see note under Task 4).

1. Predictions (Part A, made before timed experiments)

Scenario	Predicted best	Reason
Aggregate 1M-row CSV by pickup location	DuckDB	SQL aggregation with no cluster/JVM startup cost should be fastest for a single grouped query.
Filter 2M-row Parquet, return 4 columns	DuckDB	Columnar Parquet + projection pushdown suits DuckDB's vectorized engine well.
Join 2M taxi rows with small zone lookup	DuckDB	Lookup table is tiny; expect an efficient in-memory hash join with low overhead.
Scale same aggregation 100K to 2M rows	DuckDB	Expected flattest growth curve; MapReduce expected worst due to per-run process overhead.
Repeat same query several times on same data	Spark (cached)	Explicit .cache() should give the biggest, most controllable speed-up on repeats.

2. Task 4 scaling plot



MapReduce shows the steepest curve and largest fixed cost (~3.1s intercept from process/temp-directory startup per run), growing to 16.0s at 2M rows. Spark starts far lower (~0.72s at 100K) and grows more slowly, since its session/JVM startup was excluded from the timed region. DuckDB is both the lowest and flattest curve (0.12s to 0.47s), reflecting minimal per-run overhead and vectorized execution. None of the three curves scale a full 20x for a 20x increase in rows, confirming each system pays a fixed per-run cost on top of a smaller marginal per-row cost.

3. Median timings summary (seconds)

Task	Condition	MapReduce	Spark	DuckDB
Task 1	Aggregation, taxi_1m.csv	7.104	0.108	0.221
Task 2	Query A (all cols), taxi_2m.parquet	—	0.540	0.120
Task 2	Query B (4 cols), taxi_2m.parquet	—	0.383	0.051
Task 3	Join + top-5 zones, taxi_2m.parquet	—	1.228	0.084
Task 4	Aggregation @ 2M rows (largest size)	16.00	1.927	0.475

Task 5 (taxi_1m.parquet, 5 repeated runs): Spark uncached fell from 3.34s (run 1) to ~0.40-0.58s (runs 2-5); Spark cached (after a 2.99s one-time cache-build) ran 0.61s -> 0.26s across 5 runs; DuckDB repeated queries fell from 0.337s (run 1) to ~0.007-0.010s (runs 2-5).

4. Three observations that surprised us

(1) On Task 1, Spark (0.108s) was faster than DuckDB (0.221s) for the exact same 1M-row CSV aggregation, even though DuckDB was expected to win. **(2)** On Task 5, DuckDB's second run collapsed from 0.337s to 0.007s — a roughly 42x drop — far larger than Spark's uncached improvement (3.34s to ~0.40-0.58s), pointing to OS-level page-cache effects rather than any DuckDB-specific caching mechanism. **(3)** MapReduce's runtime barely changed between the warm-up run and the three recorded runs at every dataset size (e.g. 2M rows: 15.97s warm-up vs 15.90-16.05s recorded), showing that its cost is dominated by fixed per-process overhead rather than any JIT-style warm-up benefit.

5. A prediction that was wrong

We predicted DuckDB would be fastest for the plain Task 1 aggregation on CSV, reasoning that its lightweight, no-JVM SQL engine would beat Spark on a single query. In practice Spark was about 2x faster (0.108s vs 0.221s). The likely explanation is that our DuckDB query used `read_csv_auto`, which re-runs CSV type/schema auto-detection on every call, while the Spark script read the CSV with an explicit schema (header=True, inferSchema=False, with manual casts), avoiding that inference cost. This taught us that comparing 'DuckDB vs Spark' is really comparing specific query implementations, not just the underlying engines — how a CSV is read (auto-detected vs explicitly typed) can matter as much as which engine runs it.

6. Projection/filter behavior (Task 2) and join strategy (Task 3)

Task 2: Spark's formatted plan for Query B shows a `ReadSchema` with only 5 columns (vs 20 for Query A) and the same `PushedFilters` in both, confirming column pruning without losing filter pushdown. DuckDB's EXPLAIN shows an identical pattern: Query B's READ_PARQUET node lists only 4 projected columns vs ~19 for Query A, with the same two filters pushed into the scan in both cases. Since Parquet is columnar, unread columns are never pulled off disk, which is why Query B (0.051-0.383s) consistently beat Query A (0.120-0.540s) in both engines despite scanning the identical set of matching rows: the difference is I/O volume, not row count. Task 3: Spark's saved physical plan confirms a `BroadcastHashJoin Inner BuildRight` with a `BroadcastExchange` on the small zone-lookup side, rather than a shuffle-heavy sort-merge join — Spark broadcast the ~265-row lookup table to every executor and joined locally, avoiding a shuffle of the much larger 2M-row trip dataset. We did not force this with a hint; it is Spark's default choice since the lookup table is far under the auto-broadcast threshold. The plan also shows the parquet scan's `ReadSchema` was pruned to only PULocationID and fare_amount, the same projection-pushdown behavior seen in Task 2. A MapReduce implementation would instead distribute the small lookup to every mapper's local memory and emit (Borough, Zone) keyed values directly, avoiding a reduce-side shuffle of the much larger trip dataset purely to join it with a tiny table — the same broadcast-vs-shuffle trade-off Spark made automatically.

7. Task 6 — tool selection

Scenario	Choice	Justification
A. 5GB Parquet, 16GB RAM laptop, 8 SQL queries, local	DuckDB	Fits comfortably in RAM; columnar Parquet suits DuckDB's pushdown; no cluster is justified.
B. 20TB immutable logs, already distributed, nightly batch	MapReduce	Data already distributed and far exceeds one machine; fault-tolerant distributed batch is required.
C. Multi-stage joins + feature transforms + ML training on a cluster	Spark	In-memory caching across iterative stages plus native MLlib support fits this pipeline directly.
D. 30GB CSV, 32GB RAM workstation, 5 aggregates, no cluster	DuckDB	Larger than comfortably fits in RAM, but DuckDB streams/out-of-core processes it on one NVMe machine.
E. 2GB dataset, colleague wants a Spark cluster 'because Big Data'	DuckDB	2GB is trivial for one laptop; a cluster adds network/serialization overhead for no benefit.
F. 200GB, no info on format/query/memory/storage/distribution	Insufficient information	Correct tool depends entirely on the missing details (format, memory fit, distribution, query type).

8. Conclusion (max 150 words)

The factor that most influenced our choice of system, beyond raw dataset size, was **fixed per-run overhead versus true computational scaling**. MapReduce's cost was dominated by process/temp-directory startup (roughly 3s before any data was touched), making it look slow even on small inputs regardless of row count. Spark's cost depended heavily on whether its session was already warm and whether data was cached in memory across repeated queries. DuckDB's speed depended on how a query was written — explicit schemas versus auto-detected CSV parsing changed its relative ranking against Spark entirely. This means dataset size alone cannot predict which tool wins: the number and pattern of repeated queries, whether a session/connection can be reused, and how carefully a query avoids unnecessary re-inference or re-reading matter just as much as how many rows are being processed.

Note: all timings were collected on a Live-USB Linux Mint session with 8GB RAM (see machine configuration in results/timings.csv) — storage speed and available memory on this setup are likely worse than a typical installed laptop, which may inflate absolute times (especially MapReduce's per-run overhead) relative to a normal machine. Relative comparisons between the three systems should still hold.